arXiv:2410.15052v5 [cs.AI] 10 Nov 2025

GlitchMiner: Mining Glitch Tokens in Large Language Models via Gradient-based Discrete Optimization

Zihui Wu[1], **Haichang Gao**[1*], **Ping Wang**[1], **Shudong Zhang**[1], **Zhaoxiang Liu**[2, 3], **Shiguo Lian**[2, 3†]
[1]School of Computer Science and Technology, Xidian University
[2]Data Science & Artificial Intelligence Research Institute, China Unicom
[3]Unicom Data Intelligence, China Unicom
zihui@stu.xidian.edu.cn, hchgao@xidian.edu.cn, liansg@chinaunicom.cn

Abstract

Glitch tokens—inputs that trigger unpredictable or anomalous behavior in Large Language Models (LLMs)—pose significant challenges to model reliability and safety. Existing detection methods primarily rely on heuristic embedding patterns or statistical anomalies within internal representations, limiting their generalizability across different model architectures and potentially missing anomalies that deviate from observed patterns. We introduce **GlitchMiner**, an behavior-driven framework designed to identify glitch tokens by maximizing predictive entropy. Leveraging a gradient-guided local search strategy, GlitchMiner efficiently explores the discrete token space without relying on model-specific heuristics or large-batch sampling. Extensive experiments across ten LLMs from five major model families demonstrate that GlitchMiner consistently outperforms existing approaches in detection accuracy and query efficiency, providing a generalizable and scalable solution for effective glitch token discovery. Code is available at https://github.com/wooozihui/GlitchMiner.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities and are increasingly deployed in high-stakes domains such as code generation (Jiang et al. 2024; Chen et al. 2021; Nijkamp et al. 2022), healthcare (Goel et al. 2023; Wang, Ma, and Chen 2023), and education (Wang et al. 2024; Jury et al. 2024). As reliance on LLMs grows, ensuring their safety and output reliability becomes imperative.

A particularly concerning threat arises from a class of anomalous inputs known as *glitch tokens* (LessWrong Community 2023). These tokens can induce LLMs to generate unexpected, erratic, or even policy-violating outputs (Geiping et al. 2024), undermining trust in their behavior. For example, as illustrated in Figure 1, when prompted to repeat the word "Mediabestanden," Llama2-7b-chat outputs "hello world"—a semantically unrelated response. This phenomenon indicates that certain tokens can trigger unpredictable model behavior, even under simple and deterministic instructions.

In response, several studies have been proposed to detect glitch tokens (Land and Bartolo 2024a; Li et al. 2024; Zhang et al. 2024). These methods typically rely on heuristic observations or statistical patterns in token embeddings or latent representations, such as small embedding norms or localized embedding clusters. Although effective in certain scenarios, such heuristics have two main limitations. First, the reliance on specific observed statistical features may not generalize well across different architectures or training setups. Second, tokens that do not conform to these empirically observed patterns may remain undetected, potentially limiting the overall coverage and robustness of these detection methods.

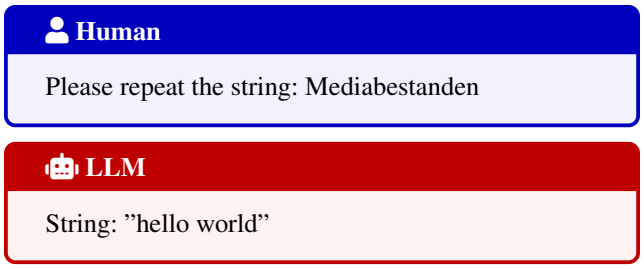

Figure 1: An illustrative example of how glitch token causing Llama2-7b-chat to fail a simple repetition task. More examples can be found in Appendix B.

To address these limitations, we propose a behavior-driven perspective. Rather than searching for fixed patterns within the model, we hypothesize that glitch tokens are outliers in the model's learned distribution, and thus elicit *uncertain* predictions. This uncertainty can be quantified via the entropy of the model's next-token distribution—high entropy indicates that the model is indecisive, providing a proxy for behavioral instability.

Building on this insight, we present **GlitchMiner**, a novel framework that identifies glitch tokens by maximizing output entropy. The core of GlitchMiner is a *gradient-guided local search*, which efficiently navigates the discrete token space to identify high-entropy candidates. Unlike prior gradient-based discrete optimization strategies that suffer from inaccurate approximations or require large batch sampling (Shin et al. 2020; Zou et al. 2023), our method restricts updates to neighboring tokens in the embedding space. This ensures accurate Taylor-based entropy estimates and improves search efficiency.

Our Contributions are summarized as follows:

*Corresponding author.
†Corresponding author.

- We propose a behavior-driven framework for glitch token detection, leveraging output entropy as an architecture-agnostic measure of model uncertainty to ensure broad applicability across diverse models.
- We introduce a gradient-guided local search algorithm that accurately estimates entropy gradients within the discrete token space, eliminating the dependence on large-batch sampling and improving search efficiency.
- Extensive experiments across 10 LLMs from 5 major model families demonstrate that GlitchMiner consistently surpasses state-of-the-art methods in detection accuracy, establishing it as a robust and effective solution for glitch token detection.

The remainder of the paper is organized as follows: Section 2 reviews related work; Section 3 describes the GlitchMiner methodology; Section 4 presents experiments and ablations; and Section 5 concludes with future directions.

2 Background and Related Work

2.1 Glitch Token Definition

In practice, the identification of glitch tokens relies on specific behavioral tests. The most common approach, used in prior work (Land and Bartolo 2024a; Li et al. 2024; Zhang et al. 2024), is the **repetition task**. In such a task, a model is prompted to repeat a given token t; a failure to accurately reproduce the token is taken as evidence of it being a glitch. This single-test method, however, is fragile. The outcome can be highly sensitive to the specific wording of the prompt, leading to inconsistent results and potential false positives.

To establish a more robust and replicable definition, we move beyond single-prompt evaluations. We classify a token t as a *glitch token* only if it fails the repetition task consistently across a diverse set of m prompt templates. This stricter criterion is formalized as follows: given a set of m templates, $h_i(\cdot)\}_{i=1}^{m}$, the failure rate for a token t is defined as:

$$\text{fail}(t) := \frac{1}{m}\sum_{i=1}^{m} \mathbb{1}\left[\arg\max_{v\in\mathcal{V}} P(v \mid h_i(t)) \neq t\right] \quad (1)$$

A token is then identified as a glitch if and only if its failure rate is absolute, i.e., $\text{fail}(t) = 1$. This rigorous cross-verification process effectively filters out template-specific artifacts, ensuring that our analysis focuses exclusively on consistently unstable tokens.

2.2 Glitch Token Detection

A series of methods have been proposed to *efficiently rank or localise* glitch tokens in large vocabularies:

Magikarp (Land and Bartolo 2024a) adopts a lightweight heuristic, screening tokens with atypically small ℓ_2 embedding norms before verifying them via the repetition task.

GlitchHunter (Li et al. 2024) observes that glitch tokens tend to cluster in embedding space. It builds a token-embedding graph and applies Leiden clustering (Traag, Waltman, and Van Eck 2019), followed by iterative hypothesis testing to refine each cluster.

GlitchProber (Zhang et al. 2024) shifts the focus from embeddings to *internal activations*. It projects hidden states and attention outputs with PCA, then trains SVM classifiers to flag activation outliers; a mitigation step masks offending neurons at inference time.

While effective, these approaches share a common limitation: they rely heavily on heuristic observations of embedding or activation patterns. Such reliance may reduce their generalizability across different architectures, and tokens not exhibiting these typical patterns may remain undetected.

In contrast, GlitchMiner employs gradient-guided discrete optimization to maximize output entropy, an architecture-agnostic indicator of predictive uncertainty that facilitates broad generalization across diverse LLMs.

2.3 Gradient-based Discrete Optimization

Gradient-based discrete optimization methods (Ebrahimi et al. 2017; Shin et al. 2020; Zou et al. 2023; Wen et al. 2024) leverage gradient information to predict how individual tokens impact the loss function. These approaches typically treat the one-hot encoding of tokens or token embeddings as continuous vectors to compute gradients, guiding token replacements for optimization.

HotFlip (Ebrahimi et al. 2017) uses the one-hot encoding of the tokens to calculate the gradients and selects the token with the largest negative gradient to replace the current token, with the goal of minimizing the loss. However, it only evaluates one candidate token per iteration, which can lead to suboptimal predictions and reduced accuracy.

AutoPrompt (Shin et al. 2020) improves upon HotFlip by evaluating multiple candidate tokens in each iteration. Instead of relying on gradients from one-hot encodings, it utilizes token embedding gradients for loss estimation, enhancing prediction accuracy by considering a broader range of potential token replacements.

GCG (Zou et al. 2023) extends HotFlip by incorporating multi-candidate token selection, similar to AutoPrompt, but it still uses the one-hot encoding of tokens to compute gradients for loss estimation. Notably, GCG has been applied to automated *jailbreaks* (Shen et al. 2023) in LLMs, efficiently searching for adversarial suffixes.

AutoPrompt and GCG both rely on **large batch sampling** to mitigate inaccuracies in gradient-based prediction. We identified that these inaccuracies arise from the inaccuracy of Taylor expansions when input tokens are distant from the original points. This overlooks a fundamental condition of Taylor approximation: its accuracy is highest for points close to the reference point.

Building on these works, we introduce a **local search strategy** in our approach. This improvement enables us to achieve high precision in gradient estimation without relying on large batch sampling, by focusing on a smaller, localized token space. By addressing the core issue of Taylor approximation accuracy, our method allows for more efficient and accurate exploration of the token space, which is particularly valuable for glitch token detection.

Algorithm 1: The GlitchMiner Pipeline

Input: LLM model $f(\cdot)$, Full vocabulary $\mathcal{T}$, Max iterations N
Output: Verified glitch token set $\mathcal{G}$
Initialization
$\mathcal{T}^* \leftarrow$ PreFilter($\mathcal{T}$) ▷ Exclude irrelevant tokens, see Sec. 3.3
$\mathcal{G} \leftarrow \emptyset$
# Iterative Mining Loop
for i from 1 to N **do**
    # 1. Gradient-guided selection of promising candidates
    $\mathcal{T}_c \leftarrow \mathcal{T} \setminus (\mathcal{T}^* \cup \mathcal{G})$ ▷ Define current candidate set
    $\mathcal{B} \leftarrow$ SelectCandidateBatch($\mathcal{T}_c$) ▷ See Sec. 3.1
    # 2. Robust verification of each candidate
    for each token $t \in \mathcal{B}$ **do**
        if VerifyIsGlitch(t) **then** ▷ See Sec. 3.2
            $\mathcal{G} \leftarrow \mathcal{G} \cup \{t\}$
        end if
        $\mathcal{T}^* \leftarrow \mathcal{T}^* \cup \{t\}$ ▷ Mark token as processed
    end for
end for
return $\mathcal{G}$

3 Methodology

Our goal is to develop a method that can automatically and efficiently discover glitch tokens in any given LLM. To this end, we propose **GlitchMiner**, a novel framework that formulates glitch token discovery as a behavior-driven optimization problem.

GlitchMiner systematically mines for these tokens through a powerful iterative loop, outlined in Algorithm 1. Each iteration first performs **Gradient-Guided Selection** to pinpoint tokens with the highest predicted output entropy. These candidates then immediately undergo **Robust Verification** to determine if they are genuine glitch tokens.

3.1 Gradient-Guided Candidate Selection

The core of GlitchMiner's discovery engine is the candidate selection process. To ensure the overall mining is efficient, this selection is not random; instead, it employs a local search strategy to intelligently find tokens that maximize model confusion, which we measure using output entropy.

Optimization Objective. To formally quantify a token's impact on model uncertainty, we calculate its output entropy within a repetition task. We use the following template for this optimization process:

User: Please repeat the string: "{token}"
Assistant: Sure, the string is: "{token}

Here, the first {token} represents the input token t being evaluated, and the second {token} is the next predicted token by LLM. To ensure that the input token appears directly as the model's next prediction, we prefill the assistant's response with the phrase *Sure, the string is: "*. The entropy is computed based on the model's next-token distribution $p(v \mid \mathbf{h}(t))$, where $\mathbf{h}(t)$ denotes the prompt generated by inserting t into

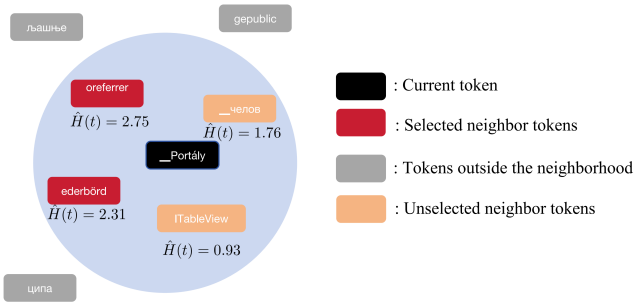

Figure 2: Visualization of GlitchMiner's local search process. The **pivot token** (black, t_c) serves as the reference point. Its **neighbor tokens** (orange and red) represent the $K = 4$ closest tokens in embedding space. Among these, the **candidate batch tokens** (red) are the top $B = 2$ tokens with the highest **approximate entropy values**, estimated via first-order Taylor approximation. Tokens outside the neighborhood (gray) are excluded to maintain approximation accuracy and computational efficiency.

the repetition template h. The entropy is thus:

$$H(t) = -\sum_{v \in \mathcal{V}} p(v \mid \mathbf{h}(t)) \log p(v \mid \mathbf{h}(t))$$

Our goal is to find a batch of tokens $\mathcal{B}$ from the current set of available tokens $\mathcal{T}_c$ that maximizes the total entropy:

$$\mathcal{B} = \arg \max_{\mathcal{B} \subset \mathcal{T}_c, |\mathcal{B}| = B} \sum_{t \in \mathcal{B}} H(t)$$

where $\mathcal{T}_c = \mathcal{T} \setminus (\mathcal{T}^* \cup \mathcal{G})$, with $\mathcal{T}^*$ being the set of filtered (see Section 3.3) or previously verified non-glitch tokens, and $\mathcal{G}$ being the set of glitch tokens found.

Local Search Strategy. To efficiently solve this optimization problem, we introduce a **local search strategy** that addresses the limitations of global Taylor approximations. This process begins with an initial pivot token t_c and iteratively refines the search. In each step:

1. A local neighborhood $\mathcal{N}_K(t_c)$ is defined, consisting of the nearest neighbors K to the current pivot token t_c in the embedding space.
2. Within this neighborhood, the entropy of each candidate token $t \in \mathcal{N}_K(t_c)$ is estimated using a first-order Taylor approximation:

$$\hat{H}(t) \approx H(t_c) + \nabla_e H(t_c)^\top (e_t - e_{t_c})$$

   where e_t and e_{t_c} are the respective embedding vectors. This approximation avoids the high cost of exact entropy calculations for all neighbors.
3. A batch $\mathcal{B}$ of B tokens with the highest approximated entropy $\hat{H}(t)$ is selected from the neighborhood. This batch is returned for verification.
4. To guide the next search step, the *actual* entropy H_t is computed for the tokens in $\mathcal{B}$, and the one with the highest actual entropy becomes the new pivot, t_c.